hackerearth

TECHNICAL INTERVIEWS

[01] THE GAP

A candidate can explain the code. Can they actually write it?

Technical interviews need to go beyond good answers.

THE ANSWER 01

“I’d use a hash map, so every lookup is O(1). Then return the matching indices.”

✓ SOUNDS RIGHT

THE CODE 02

```
1 def two_sum(nums, target):
2
3
4
```

NOT YET SHOWN

[02] THE PROBLEM

Knowing the right answer isn't the same as solving the problem.

A candidate may explain a concept well.

But can they debug an issue, handle edge cases or justify a design decision?

DEBUG 01

Debug an issue

```
for i in range(n):
  total += a[i+1]
```

IndexError

EDGE CASES 02

Handle edge cases

```
nums = []
target = None
n = 10**9
```

DESIGN 03

Justify a design decision

```
# cache or db?
# sync or queue?
why?
```

[03] WHAT TO EVALUATE

Look beyond the final answer.

Evaluate how candidates:

01 Approach a problem ?→

02 Write and debug code </>

03 Handle edge cases [ ]

04 Explain technical decisions //

05 The final answer =

[04] INSIDE ONSCREEN

Ask questions that go deeper.

HackerEarth OnScreen supports adaptive technical interviews, live coding, system design and follow-up questions that probe a candidate's reasoning.

- Adaptive technical interviews
- Live coding
- System design
- Follow-up questions

THE GOAL

Evaluate demonstrated ability, not just recall.

[05] NEXT STEP

Technical hiring deserves more than a convincing conversation.

Build your evaluation around the skills the role actually requires.

Explore HackerEarth OnScreen →